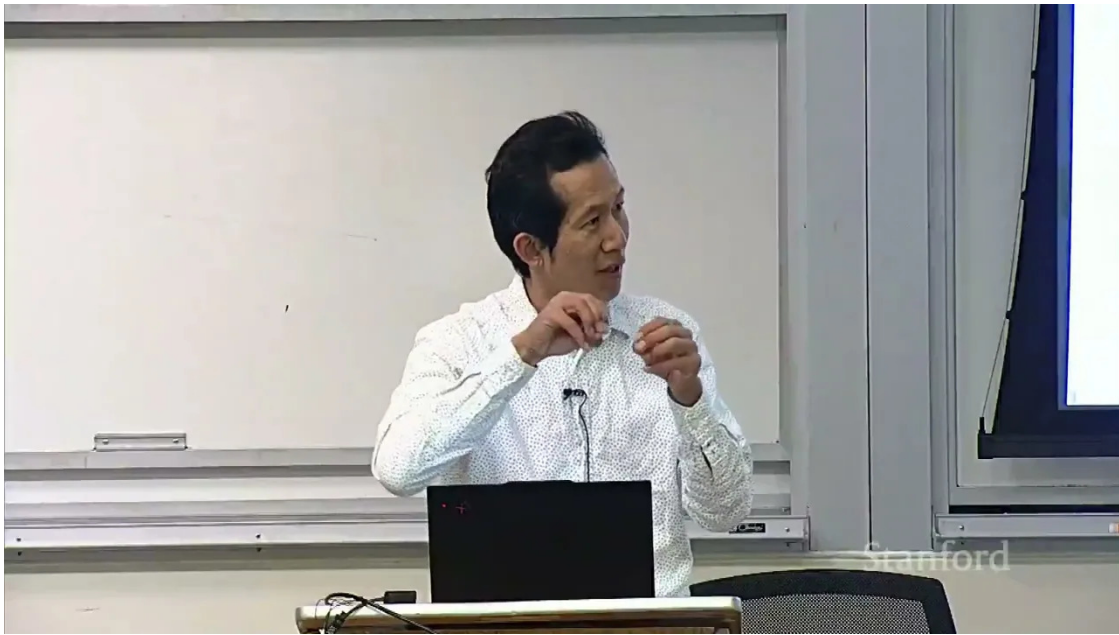


Stanford CS336: Language Modeling from Scratch

Lecture 7: Parallelism 1

Spring 2026 中文精讲笔记

主讲: Percy Liang



频道: Stanford Online

发布日期: 2026-04-28

时长: 1:21:02

视频: <https://www.youtube.com/watch?v=Szp0cwdILOY>

官方主页: <https://cs336.stanford.edu/>

目录

1 导读：问题、证据与阅读方法	2
2 并行训练的成本模型	2
3 硬件拓扑与集合通信	7
4 环形 all-reduce	11
5 数据并行与 DDP	15
6 张量并行	19
7 序列并行	23
8 流水线并行	27
9 组合并行策略	31
10 代码化复现：最小机制	35
11 总结与延伸	36
12 资料来源与审计边界	36

1 导读：问题、证据与阅读方法

本讲主问题

当显卡显存和计算不再足够时，应怎样切分数据、参数、激活和层，同时控制通信与同步成本？

并行策略的本质是决定哪一维被切分、哪一份状态被复制、在哪个边界执行哪一种集合通信。本笔记以课程视频、人工字幕和 Stanford 官方讲义或代码为共同证据源。图像均来自每十五秒抽帧后的人工接触表审查，时间脚注可回到原视频复核；公式区分课程机制关系和本笔记的决策分析框架。学习时建议先读主问题，再沿“机制-资源-失败模式-验证指标”四列整理自己的实验。

首先排除的误读

只看显存节省会忽略通信；只看理论带宽又会忽略启动延迟、拓扑不均匀和流水线气泡。

1.1 关键数字核对

主张	数值或关系	视频时间
进程总数	world size p	00:06:00
环形 all-reduce 阶段数	$2(p-1)$	00:18:00
数据并行本地 batch	B_{global}/p	00:35:00
流水线气泡项	$p-1$ 个阶段间隙	01:02:00

1.2 本章小结

本章建立证据边界：视频解释提供因果和强调，官方材料提供公式与代码，精选帧提供可核查的视觉状态。后文每个主题都同时回答“为何重要、怎样工作、怎样失败、怎样验证”，避免把课程压缩成术语目录。

2 并行训练的成本模型

本节路线

本节依次处理：分布式训练并行化的目标和成本模型、节点内外 GPU 互连带宽与延迟层级、broadcast reduce gather scatter 等通信原语、all-reduce 汇总并回传分片结果。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$T_{\text{comm}} = \alpha + \beta n$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

$$G_1 = \frac{B_1}{C_1 + \lambda R_1 + \epsilon}$$

- B_1 是本节方法带来的质量、吞吐或覆盖收益。
- C_1 是新增计算、显存、通信、数据或标注成本。
- R_1 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_1 \leq C_{\max}, \quad R_1 \leq R_{\max}, \quad Q_1 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

2.1 分布式训练并行化的目标和成本模型

核心判断。“分布式训练并行化的目标和成本模型”是“并行训练的成本模型”中的一个可检验设计选择，而不是只需记忆的产品名。把该主题拆成输入、变换、持久状态和输出四部分，再分别记录直接机制指标、资源成本和最终质量；若中间量不按预测变化，应优先检查实现和数据边界。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 1 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“分布式训练并行化的目标和成本模型”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

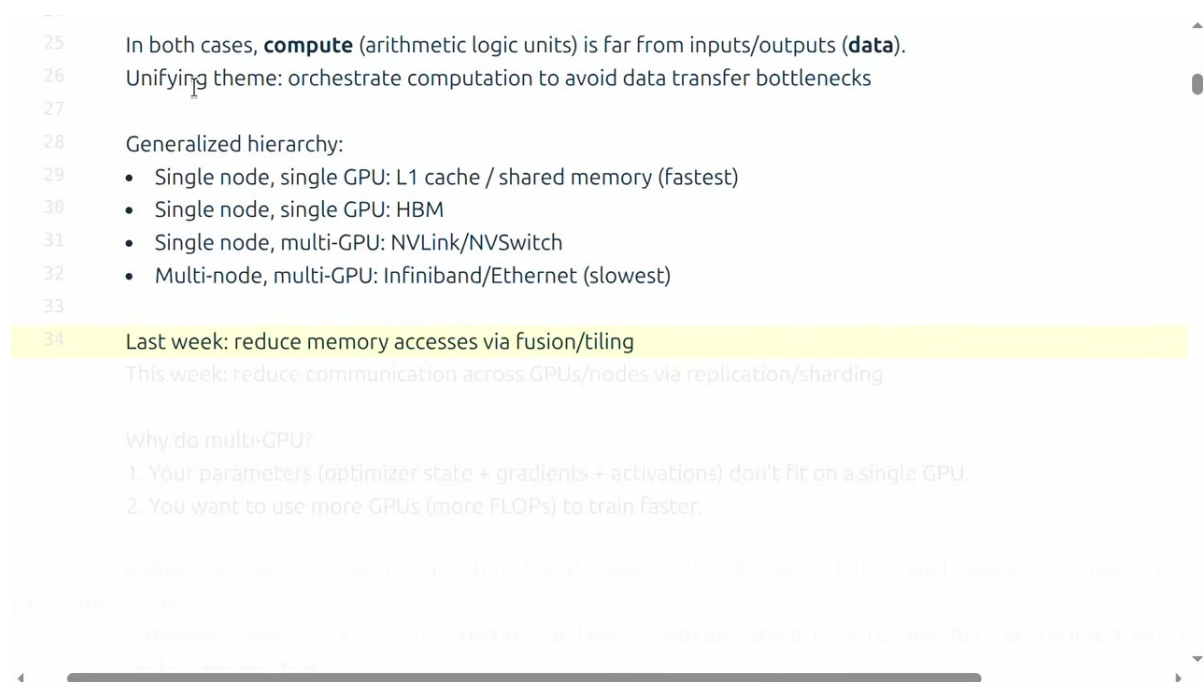


图 1: 分布式训练并行化的目标和成本模型¹

2.2 节点内外 GPU 互连带宽与延迟层级

核心判断。“节点内外 GPU 互连带宽与延迟层级”是“并行训练的成本模型”中的一个可检验设计选择，而不是只需记忆的产品名。把算子映射到线程或矩阵单元，并明确数据位于寄存器、片上 SRAM 还是 HBM；瓶颈判断依赖算术强度，而不是只比较峰值 FLOP/s。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 2 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“节点内外 GPU 互连带宽与延迟层级”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹视频画面时间区间：00:02:45–00:03:00。

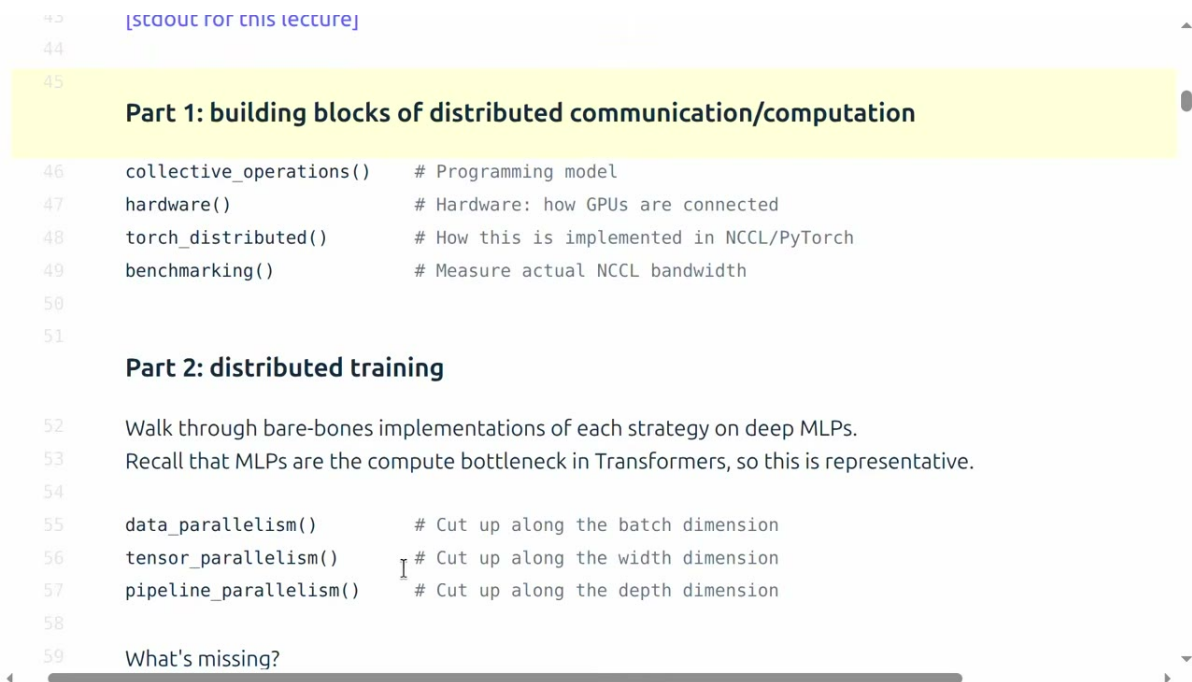


图 2: 节点内外 GPU 互连带宽与延迟层级²

2.3 broadcast reduce gather scatter 等通信原语

核心判断。“broadcast reduce gather scatter 等通信原语”是“并行训练的成本模型”中的一个可检验设计选择，而不是只需记忆的产品名。把该主题拆成输入、变换、持久状态和输出四部分，再分别记录直接机制指标、资源成本和最终质量；若中间量不按预测变化，应优先检查实现和数据边界。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 3 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

常见失效模式

围绕“broadcast reduce gather scatter 等通信原语”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

²视频画面时间区间：00:05:30–00:05:45。

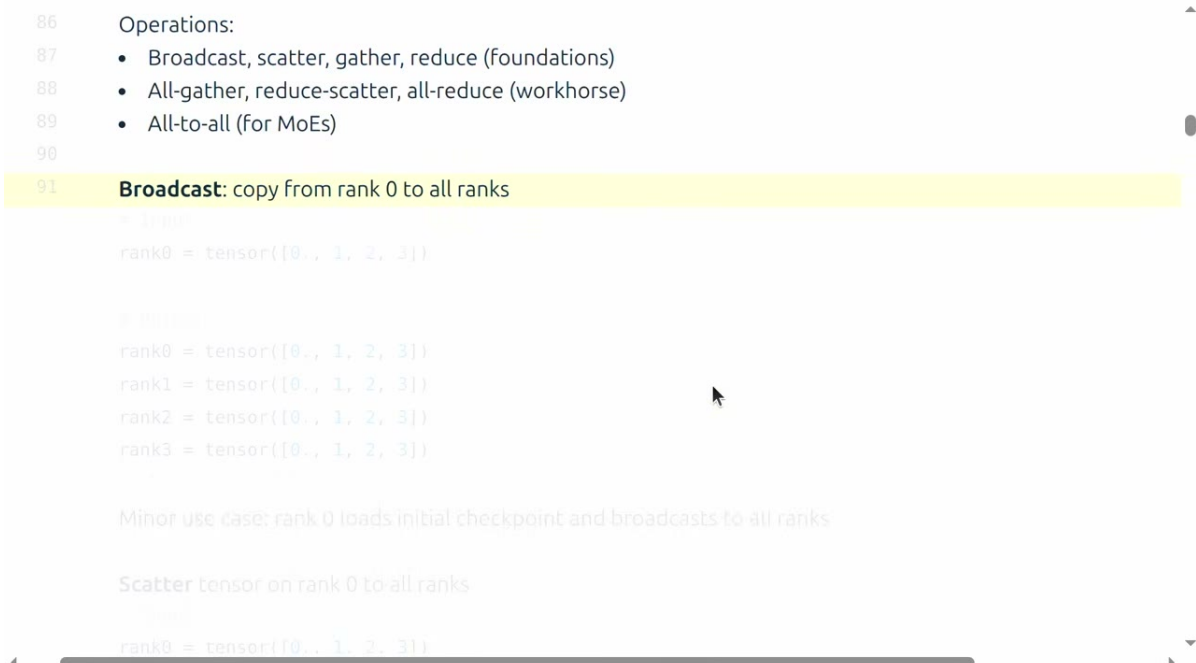


图 3: broadcast reduce gather scatter 等通信原语³

2.4 all-reduce 汇总并回传分片结果

核心判断。“all-reduce 汇总并回传分片结果”是“并行训练的成本模型”中的一个可检验设计选择，而不是只需记忆的产品名。每种并行都要列出切分维、复制状态和通信原语；端到端收益由通信是否可隐藏、链路拓扑和负载均衡决定。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 4 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“all-reduce 汇总并回传分片结果”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

³视频画面时间区间：00:08:15–00:08:30。


```
122 # Output
123 rank0 = tensor([0., 1, 2, 3])
124
125 Note: stepping stone to understanding all-gather
126
127 Reduce pieces from all ranks to rank 0, applying some operation (e.g., sum, min, max)
128 # Input
129 rank0 = tensor([0.])
130 rank1 = tensor([1.])
131 rank2 = tensor([2.])
132 rank3 = tensor([3.])
133
134 # Output
135 rank0 = tensor([6.]) # Sum of all ranks (0 + 1 + 2 + 3)
```

Note: stepping stone to understanding all-reduce

All-gather: perform gather to all ranks, not just rank 0

rank0 = tensor([0.])

图 4: all-reduce 汇总并回传分片结果⁴

2.5 本章小结

“并行训练的成本模型”的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。只看显存节省会忽略通信；只看理论带宽又会忽略启动延迟、拓扑不均匀和流水线气泡。

3 硬件拓扑与集合通信

本节路线

本节依次处理：环形 all-reduce 的 reduce-scatter 与 all-gather、通信量带宽与启动延迟组成的成本模型、分布式进程组与 rank 的程序接口。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$V_{\text{ring}} = 2 \frac{p-1}{p} n$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

⁴视频画面时间区间：00:11:15–00:11:30。

$$G_2 = \frac{B_2}{C_2 + \lambda R_2 + \epsilon}$$

- B_2 是本节方法带来的质量、吞吐或覆盖收益。
- C_2 是新增计算、显存、通信、数据或标注成本。
- R_2 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_2 \leq C_{\max}, \quad R_2 \leq R_{\max}, \quad Q_2 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

3.1 环形 all-reduce 的 reduce-scatter 与 all-gather

核心判断。“环形 all-reduce 的 reduce-scatter 与 all-gather”是“硬件拓扑与集合通信”中的一个可检验设计选择，而不是只需记忆的产品名。每种并行都要列出切分维、复制状态和通信原语；端到端收益由通信是否可隐藏、链路拓扑和负载均衡决定。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 5 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“环形 all-reduce 的 reduce-scatter 与 all-gather”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

```
148 rank1 = tensor([0., 1, 2, 3])
149 rank2 = tensor([0., 1, 2, 3])
150 rank3 = tensor([0., 1, 2, 3])
151
152 Use case: each rank holds parameter shard, gather to get full parameters for forward pass
153
154 Reduce-scatter: perform reduce on each dimension, scatter results
155 # Input
156 rank0 = tensor([0., 1, 2, 3])
157 rank1 = tensor([1., 2, 3, 4])
158 rank2 = tensor([2., 3, 4, 5])
159 rank3 = tensor([3., 4, 5, 6])
160
161 # Output
162 rank0 = tensor([6.]) # Sum along dim 0 (0 + 1 + 2 + 3)
163 rank1 = tensor([10.]) # Sum along dim 1 (1 + 2 + 3 + 4)
164 rank2 = tensor([14.]) # Sum along dim 2 (2 + 3 + 4 + 5)
165 rank3 = tensor([18.]) # Sum along dim 3 (3 + 4 + 5 + 6)
166
167 Use case: after backward pass, sum gradients from different data shards, but distribute storage
```

图 5: 环形 all-reduce 的 reduce-scatter 与 all-gather⁵

3.2 通信量带宽与启动延迟组成的成本模型

核心判断。“通信量带宽与启动延迟组成的成本模型”是“硬件拓扑与集合通信”中的一个可检验设计选择，而不是只需记忆的产品名。prefill 可以沿序列并行而 decode 逐 token 串行；服务评估要同时报告首 token、token 间隔、吞吐、尾延迟和显存峰值。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 6 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“通信量带宽与启动延迟组成的成本模型”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

⁵视频画面时间区间：00:14:00–00:14:15。

```

176 # Output
177 rank0 = tensor([6., 10, 14, 18])
178 rank1 = tensor([6., 10, 14, 18])
179 rank2 = tensor([6., 10, 14, 18])
180 rank3 = tensor([6., 10, 14, 18])
181
182 Use case: after backward pass, sum gradients from different data shards, but replicate full parameters
183 Breaking all-reduce into reduce-scatter + all-gather allows for flexibility (e.g., ZeRO/FSDP)
184
185 All-to-all: each rank sends each other rank some tensor (most general)
186
187 # Input
188 rank0 = tensor([0., 1, 2, 3]) # send 0 to rank 0, 1 to rank 1, 2 to rank 2, 3 to rank 3
189 rank1 = tensor([4., 5, 6, 7]) # send 4 to rank 0, 5 to rank 1, 6 to rank 2, 7 to rank 3
190 rank2 = tensor([8., 9, 10, 11]) # send 8 to rank 0, 9 to rank 1, 10 to rank 2, 11 to rank 3
191 rank3 = tensor([12., 13, 14, 15]) # send 12 to rank 0, 13 to rank 1, 14 to rank 2, 15 to rank 3
192
193 # Output
194 rank0 = tensor([0, 4, 8, 12])
195 rank1 = tensor([1, 5, 9, 13])
196 rank2 = tensor([2, 6, 10, 14])
197 rank3 = tensor([3, 7, 11, 15])

```

图 6: 通信量带宽与启动延迟组成的成本模型⁶

3.3 分布式进程组与 rank 的程序接口

核心判断。“分布式进程组与 rank 的程序接口”是“硬件拓扑与集合通信”中的一个可检验设计选择，而不是只需记忆的产品名。把该主题拆成输入、变换、持久状态和输出四部分，再分别记录直接机制指标、资源成本和最终质量；若中间量不按预测变化，应优先检查实现和数据边界。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 7 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

常见失效模式

围绕“分布式进程组与 rank 的程序接口”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

⁶视频画面时间区间：00:16:45–00:17:00。

```

186 # Input
187 rank0 = tensor([0., 1, 2, 3]) # send 0 to rank 0, 1 to rank 1, 2 to rank 2, 3 to rank 3
188 rank1 = tensor([4., 5, 6, 7]) # send 4 to rank 0, 5 to rank 1, 6 to rank 2, 7 to rank 3
189 rank2 = tensor([8., 9, 10, 11]) # send 8 to rank 0, 9 to rank 1, 10 to rank 2, 11 to rank 3
190 rank3 = tensor([12., 13, 14, 15]) # send 12 to rank 0, 13 to rank 1, 14 to rank 2, 15 to rank 3
191
192 # Output
193 rank0 = tensor([0, 4, 8, 12])
194 rank1 = tensor([1, 5, 9, 13])
195 rank2 = tensor([2, 6, 10, 14])
196 rank3 = tensor([3, 7, 11, 15])
197
198 Notes:
199 • Useful for MoEs: each rank has split of data and subset of experts; need to route data to experts
200 • For balanced splits, all-to-all looks like transpose
201 • Also handles unbalanced splits (but want splits to be as balanced as possible)
202
203 Way to remember the terminology:
204 • Reduce: performs some associative/commutative operation (sum, min, max)
205
206 - Gather is inverse of Scatter

```

图 7: 分布式进程组与 rank 的程序接口⁷

3.4 本章小结

“硬件拓扑与集合通信”的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。只看显存节省会忽略通信；只看理论带宽又会忽略启动延迟、拓扑不均匀和流水线气泡。

4 环形 all-reduce

本节路线

本节依次处理：集合通信的可执行代码示例、服务器与集群拓扑影响通信路径、数据并行复制参数并切分样本。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$g = \frac{1}{p} \sum_{r=1}^p g_r$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

⁷视频画面时间区间：00:19:30–00:19:45。

$$G_3 = \frac{B_3}{C_3 + \lambda R_3 + \epsilon}$$

- B_3 是本节方法带来的质量、吞吐或覆盖收益。
- C_3 是新增计算、显存、通信、数据或标注成本。
- R_3 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_3 \leq C_{\max}, \quad R_3 \leq R_{\max}, \quad Q_3 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

4.1 集合通信的可执行代码示例

核心判断。“集合通信的可执行代码示例”是“环形 all-reduce”中的一个可检验设计选择，而不是只需记忆的产品名。每种并行都要列出切分维、复制状态和通信原语；端到端收益由通信是否可隐藏、链路拓扑和负载均衡决定。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 8 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“集合通信的可执行代码示例”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

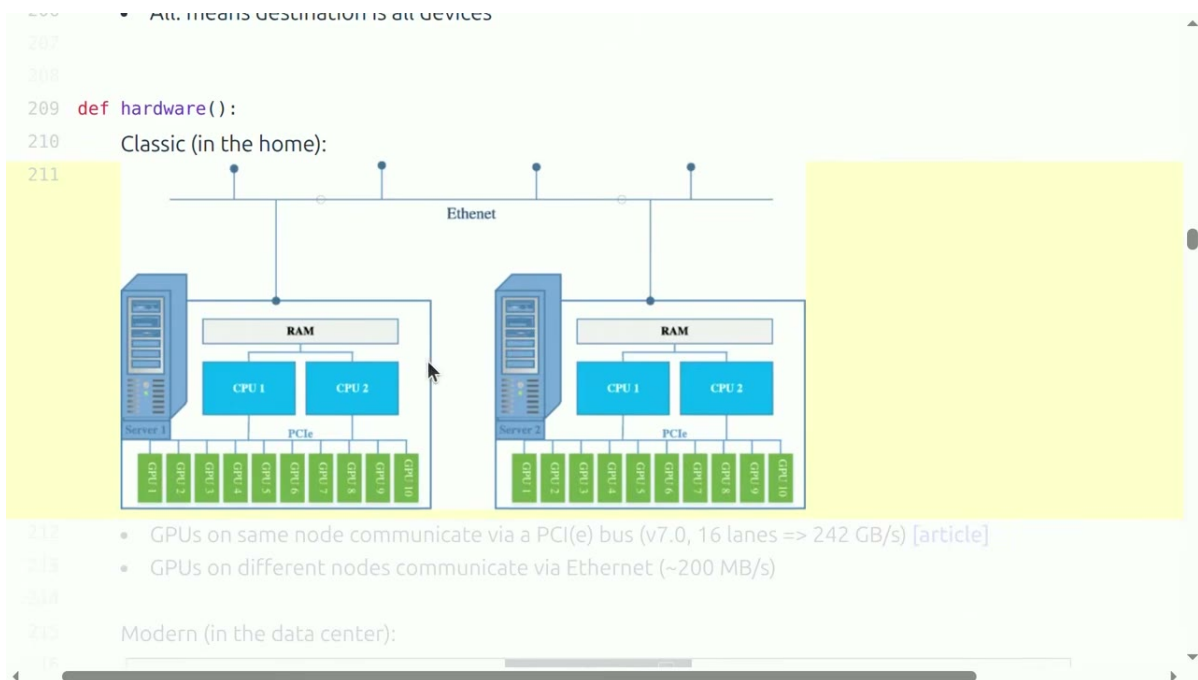


图 8: 集合通信的可执行代码示例⁸

4.2 服务器与集群拓扑影响通信路径

核心判断。“服务器与集群拓扑影响通信路径”是“环形 all-reduce”中的一个可检验设计选择，而不是只需记忆的产品名。把该主题拆成输入、变换、持久状态和输出四部分，再分别记录直接机制指标、资源成本和最终质量；若中间量不按预测变化，应优先检查实现和数据边界。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 9 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“服务器与集群拓扑影响通信路径”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

⁸视频画面时间区间：00:22:15–00:22:30。

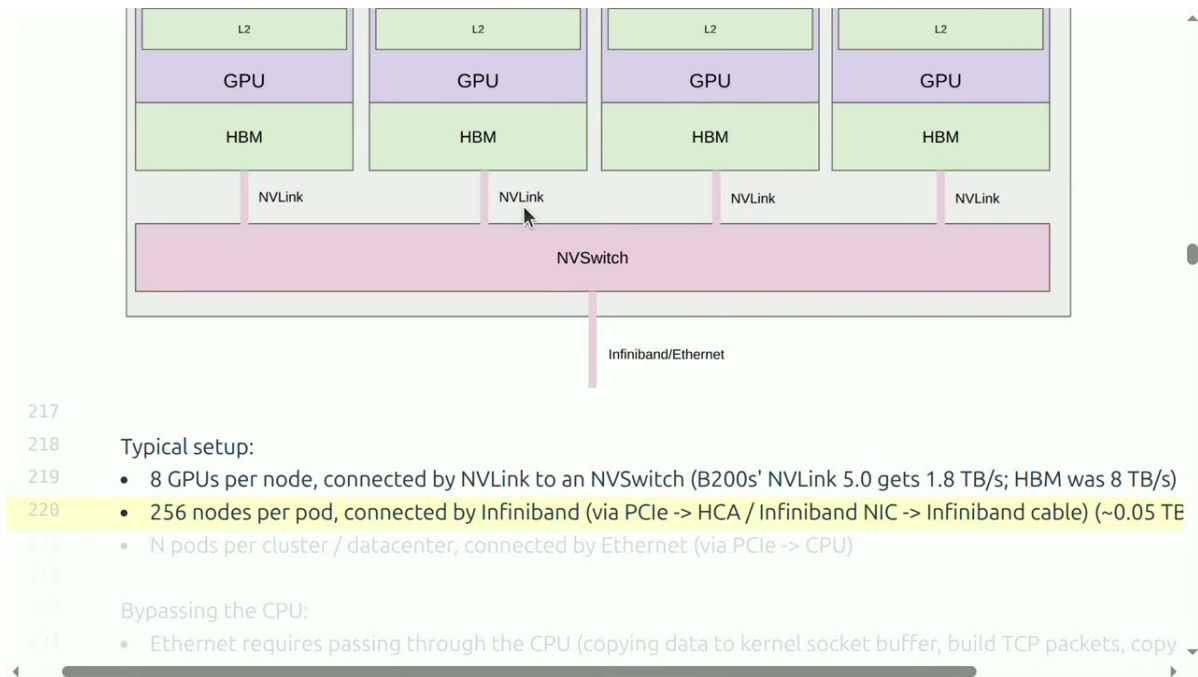


图 9: 服务器与集群拓扑影响通信路径⁹

4.3 数据并行复制参数并切分样本

核心判断。“数据并行复制参数并切分样本”是“环形 all-reduce”中的一个可检验设计选择，而不是只需记忆的产品名。每种并行都要列出切分维、复制状态和通信原语；端到端收益由通信是否可隐藏、链路拓扑和负载均衡决定。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 10 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

常见失效模式

围绕“数据并行复制参数并切分样本”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

⁹视频画面时间区间：00:25:00–00:25:15。

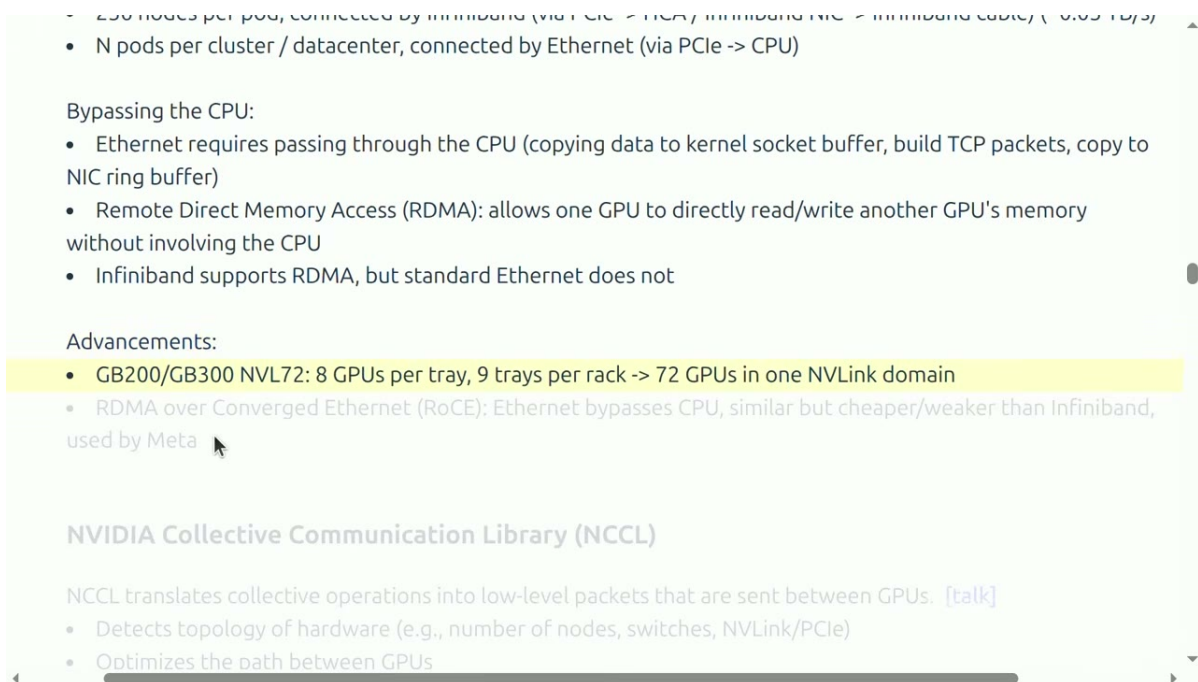


图 10: 数据并行复制参数并切分样本¹⁰

4.4 本章小结

“环形 all-reduce” 的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。只看显存节省会忽略通信；只看理论带宽又会忽略启动延迟、拓扑不均匀和流水线气泡。

5 数据并行与 DDP

本节路线

本节依次处理：数据并行中的梯度聚合时序、DDP 梯度桶与反向计算重叠、模型并行在参数维度切分算子。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$B_{\text{local}} = \frac{B_{\text{global}}}{p}$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

¹⁰ 视频画面时间区间：00:27:45–00:28:00。

$$G_4 = \frac{B_4}{C_4 + \lambda R_4 + \epsilon}$$

- B_4 是本节方法带来的质量、吞吐或覆盖收益。
- C_4 是新增计算、显存、通信、数据或标注成本。
- R_4 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_4 \leq C_{\max}, \quad R_4 \leq R_{\max}, \quad Q_4 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

5.1 数据并行中的梯度聚合时序

核心判断。“数据并行中的梯度聚合时序”是“数据并行与 DDP”中的一个可检验设计选择，而不是只需记忆的产品名。每种并行都要列出切分维、复制状态和通信原语；端到端收益由通信是否可隐藏、链路拓扑和负载均衡决定。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 11 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“数据并行中的梯度聚合时序”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

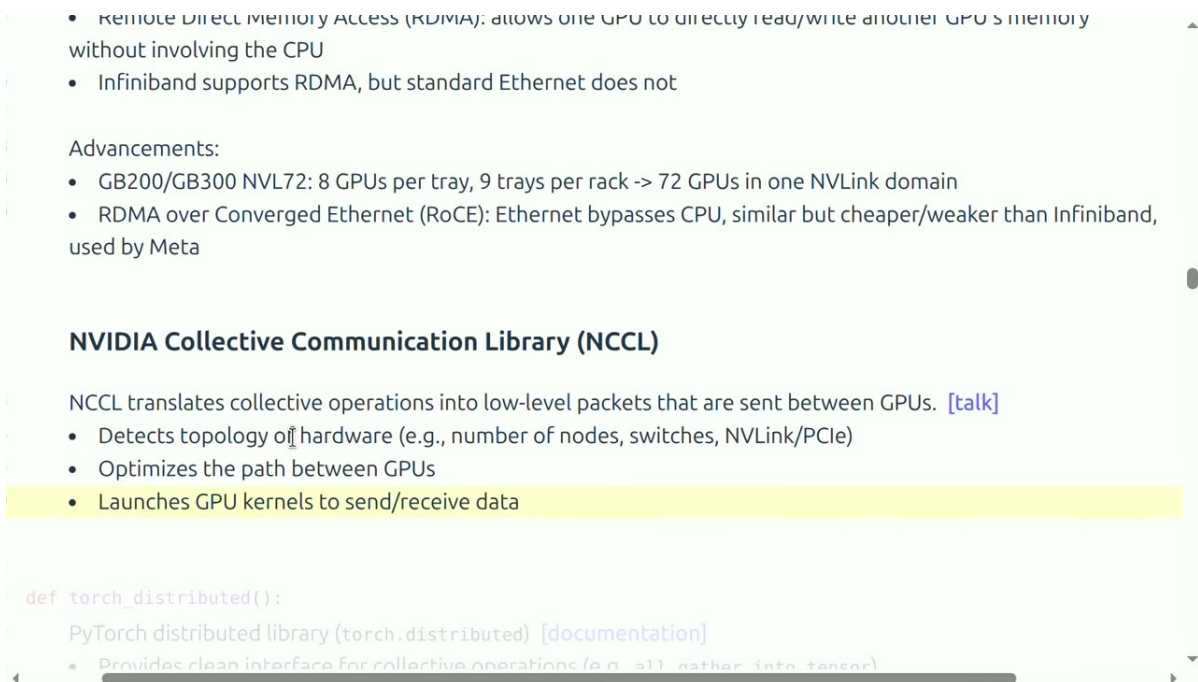


图 11: 数据并行中的梯度聚合时序¹¹

5.2 DDP 梯度桶与反向计算重叠

核心判断。“DDP 梯度桶与反向计算重叠”是“数据并行与 DDP”中的一个可检验设计选择，而不是只需记忆的产品名。把该主题拆成输入、变换、持久状态和输出四部分，再分别记录直接机制指标、资源成本和最终质量；若中间量不按预测变化，应优先检查实现和数据边界。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 12 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“DDP 梯度桶与反向计算重叠”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹¹视频画面时间区间：00:30:45–00:31:00。

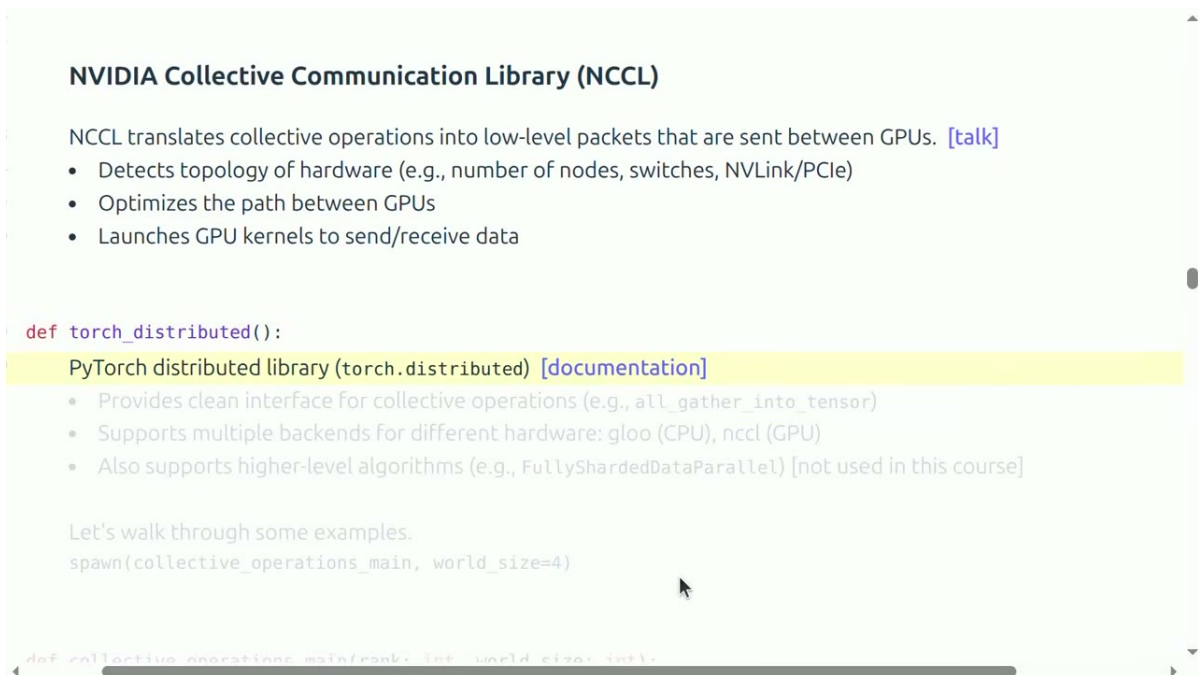


图 12: DDP 梯度桶与反向计算重叠¹²

5.3 模型并行在参数维度切分算子

核心判断。“模型并行在参数维度切分算子”是“数据并行与 DDP”中的一个可检验设计选择，而不是只需记忆的产品名。把该主题拆成输入、变换、持久状态和输出四部分，再分别记录直接机制指标、资源成本和最终质量；若中间量不按预测变化，应优先检查实现和数据边界。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 13 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

常见失效模式

围绕“模型并行在参数维度切分算子”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹²视频画面时间区间：00:36:15–00:36:30。

```
247 # Provides clean interface for collective operations (e.g., all_gather_into_tensor)
248 # Supports multiple backends for different hardware: gloo (CPU), nccl (GPU)
249 # Also supports higher-level algorithms (e.g., FullyShardedDataParallel) [not used in this course]
250
251 rank = 0
252 world_size = 4
253
254 Let's walk through some examples.
255 spawn(collective_operations_main, world_size=4)
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
```

图 13: 模型并行在参数维度切分子¹³

5.4 本章小结

“数据并行与 DDP” 的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。只看显存节省会忽略通信；只看理论带宽又会忽略启动延迟、拓扑不均匀和流水线气泡。

6 张量并行

本节路线

本节依次处理：MLP 列并行和行并行的组合、注意力头在张量并行设备间切分、张量并行层间的 all-reduce 通信。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$Y = X[W_1 W_2 \cdots W_p]$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

¹³视频画面时间区间：00:39:00–00:39:15。

$$G_5 = \frac{B_5}{C_5 + \lambda R_5 + \epsilon}$$

- B_5 是本节方法带来的质量、吞吐或覆盖收益。
- C_5 是新增计算、显存、通信、数据或标注成本。
- R_5 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_5 \leq C_{\max}, \quad R_5 \leq R_{\max}, \quad Q_5 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

6.1 MLP 列并行和行并行的组合

核心判断。“MLP 列并行和行并行的组合”是“张量并行”中的一个可检验设计选择，而不是只需记忆的产品名。把该主题拆成输入、变换、持久状态和输出四部分，再分别记录直接机制指标、资源成本和最终质量；若中间量不按预测变化，应优先检查实现和数据边界。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 14 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“MLP 列并行和行并行的组合”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。


```
Let's walk through some examples.

spawn(collective_operations_main, world_size=4)

rank = 0
world_size = 4

def collective_operations_main(rank: int, world_size: int):
    """This function is running asynchronously for each process (rank = 0, ..., world_size)"""
    setup(rank, world_size)

    ### All-reduce (dist = torch.distributed)
    dist.barrier() # Waits for all processes to get to this point (in this case, for print)

    data = tensor([0., 1, 2, 3], device=cuda_if_available(rank)) + rank # Both input and
    print(f"Rank {rank} [before all-reduce]: {data}", flush=True)
    dist.all_reduce(tensor=data, op=dist.ReduceOp.SUM, async_op=False) # Modifies tensor
    print(f"Rank {rank} [after all-reduce]: {data}", flush=True)

    ### Reduce-scatter
```

图 14: MLP 列并行和行并行的组合¹⁴

6.2 注意力头在张量并行设备间切分

核心判断。“注意力头在张量并行设备间切分”是“张量并行”中的一个可检验设计选择，而不是只需记忆的产品名。先区分 query 侧计算、历史 key/value 状态和完整注意力矩阵；核算时同时写出序列长度依赖与缓存字节数，验证时观察有效带宽、核函数时间和质量变化。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 15 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“注意力头在张量并行设备间切分”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹⁴视频画面时间区间：00:41:45–00:42:00。


```
242 • Supports multiple backends for different hardware: gloo (CPU), nccl (GPU)
    • Also supports higher-level algorithms (e.g., FullyShardedDataParallel) [not used in this course]
rank = 0
world_size = 4
spawn(collective_operations_main, world_size=4)

248
249 def collective_operations_main(rank: int, world_size: int):
250     """This function is running asynchronously for each process (rank = 0, ..., world_size - 1)."""
251     setup(rank, world_size)
252
253     ### All-reduce (dist = torch.distributed)
254     dist.barrier() # Waits for all processes to get to this point (in this case, for print statements)
255
256     data = tensor([0., 1, 2, 3], device=cuda_if_available(rank)) + rank # Both input and output
257
258     print(f"Rank {rank} [before all-reduce]: {data}", flush=True)
259     dist.all_reduce(tensor=data, op=dist.ReduceOp.SUM, async_op=False) # Modifies tensor in place
260     print(f"Rank {rank} [after all-reduce]: {data}", flush=True)
261
262     ### Reduce-scatter
263     dist.barrier()
```

图 15: 注意力头在张量并行设备间切分¹⁵

6.3 张量并行层间的 all-reduce 通信

核心判断。“张量并行层间的 all-reduce 通信”是“张量并行”中的一个可检验设计选择，而不是只需记忆的产品名。每种并行都要列出切分维、复制状态和通信原语；端到端收益由通信是否可隐藏、链路拓扑和负载均衡决定。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 16 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

常见失效模式

围绕“张量并行层间的 all-reduce 通信”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹⁵视频画面时间区间：00:44:30–00:44:45。

```

299 [How to reason about collective operations]
300 [Sample benchmarking code]
301
302 def all_reduce(rank: int, world_size: int, num_elements: int):
303     setup(rank, world_size)
304
305     # Create tensor
306     data = torch.randn(num_elements, device=cuda_if_available(rank))
307
308     # Warmup
309     dist.all_reduce(tensor=data, op=dist.ReduceOp.SUM, async_op=False)
310     torch.cuda.synchronize() # Wait for CUDA kernels to finish
311     dist.barrier()           # Wait for all the processes to get here
312
313     # Perform all-reduce
314     start_time = time.time()
315     dist.all_reduce(tensor=data, op=dist.ReduceOp.SUM, async_op=False)
316     torch.cuda.synchronize() # Wait for CUDA kernels to finish
317     dist.barrier()           # Wait for all the processes to get here
318     end_time = time.time()
319
320

```

图 16: 张量并行层间的 all-reduce 通信¹⁶

6.4 本章小结

“张量并行”的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。只看显存节省会忽略通信；只看理论带宽又会忽略启动延迟、拓扑不均匀和流水线气泡。

7 序列并行

本节路线

本节依次处理：序列并行切分逐 token 激活、流水线并行把层分配到不同阶段、流水线气泡与 microbatch 数量。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$Y = \sum_{r=1}^p X_r W_r$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

¹⁶视频画面时间区间：00:47:15–00:47:30。

$$G_6 = \frac{B_6}{C_6 + \lambda R_6 + \varepsilon}$$

- B_6 是本节方法带来的质量、吞吐或覆盖收益。
- C_6 是新增计算、显存、通信、数据或标注成本。
- R_6 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_6 \leq C_{\max}, \quad R_6 \leq R_{\max}, \quad Q_6 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

7.1 序列并行切分逐 token 激活

核心判断。“序列并行切分逐 token 激活”是“序列并行”中的一个可检验设计选择，而不是只需记忆的产品名。每种并行都要列出切分维、复制状态和通信原语；端到端收益由通信是否可隐藏、链路拓扑和负载均衡决定。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 17 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“序列并行切分逐 token 激活”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

```

dist.all_reduce(tensor=data, op=dist.ReduceOp.SUM, async_op=False)
torch.cuda.synchronize() # Wait for CUDA kernels to finish
dist.barrier()           # Wait for all the processes to get here
end_time = time.time()

duration = end_time - start_time
print(f"[all_reduce] Rank {rank}: all_reduce(world_size={world_size}, num_elements={num_elements}) took
r_duration(duration))", flush=True)

# Measure the effective bandwidth
dist.barrier()
size_bytes = data.element_size() * data.numel()
sent_bytes = size_bytes * 2 * (world_size - 1) # 2x because send + receive, world_size-1 steps in all-reduce
total_duration = world_size * duration
bandwidth = sent_bytes / total_duration
print(f"[all_reduce] Rank {rank}: all_reduce measured bandwidth = {round(bandwidth / 1024*3)} GB/s", flush=True)

# Notes:
# - Effective bandwidth ~ 2 * size_bytes / total_duration
# - Independent of world_size
# - Independent of topology (ring or tree)

```

图 17: 序列并行切分逐 token 激活¹⁷

7.2 流水线并行把层分配到不同阶段

核心判断。“流水线并行把层分配到不同阶段”是“序列并行”中的一个可检验设计选择，而不是只需记忆的产品名。每种并行都要列出切分维、复制状态和通信原语；端到端收益由通信是否可隐藏、链路拓扑和负载均衡决定。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 18 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“流水线并行把层分配到不同阶段”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹⁷视频画面时间区间：00:50:00–00:50:15。

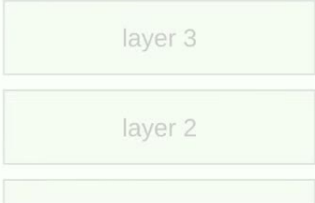
```
363 sent_bytes = data_bytes * (world_size - 1) # How much needs to be sent (no 2x here)
364 total_duration = world_size * duration # Total time for transmission
365 bandwidth = sent_bytes / total_duration
366 print(f"[reduce_scatter] Rank {rank}: reduce_scatter measured bandwidth = {round(bandwidth / 1024**3)} GB/s",
flush=True)
367
368 # Notes:
369 # - all-reduce = reduce-scatter + all-gather
370 # - all-reduce moves 2x the data in 2x the time compared to reduce-scatter, so similar bandwidth
371
372 cleanup()
373
374
375 def data_parallelism():
376

```

图 18: 流水线并行把层分配到不同阶段¹⁸

7.3 流水线气泡与 microbatch 数量

核心判断。“流水线气泡与 microbatch 数量”是“序列并行”中的一个可检验设计选择，而不是只需记忆的产品名。每种并行都要列出切分维、复制状态和通信原语；端到端收益由通信是否可隐藏、链路拓扑和负载均衡决定。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 19 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

常见失效模式

围绕“流水线气泡与 microbatch 数量”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹⁸视频画面时间区间：00:53:00–00:53:15。

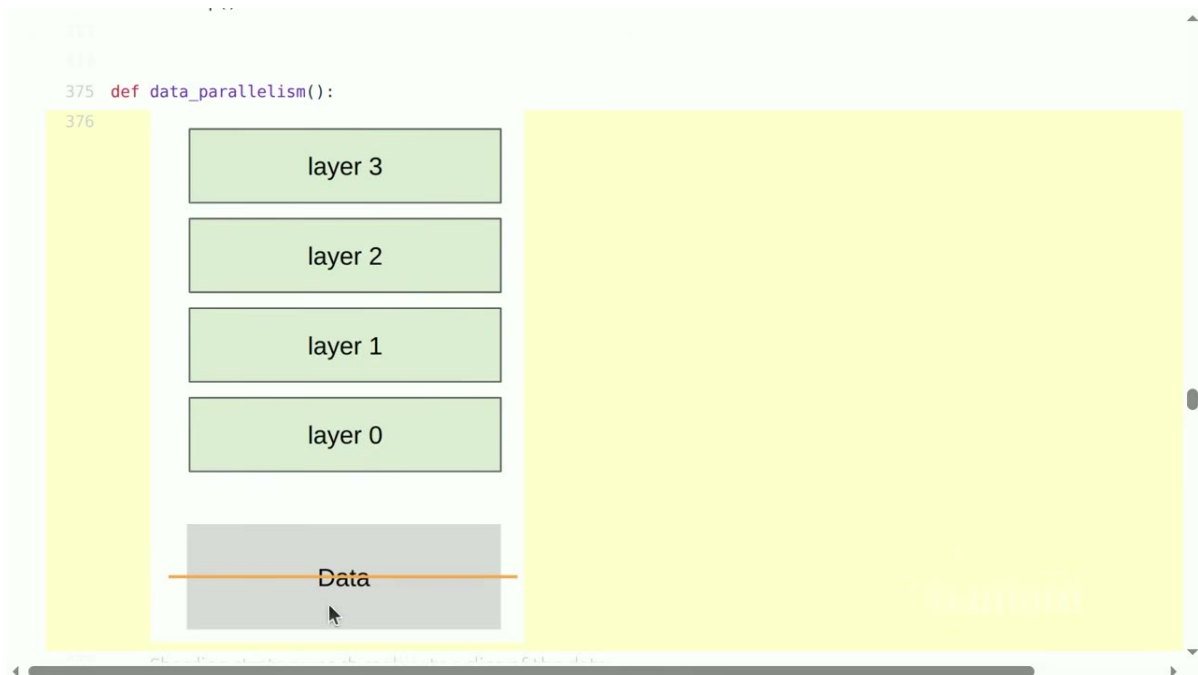


图 19: 流水线气泡与 microbatch 数量¹⁹

7.4 本章小结

“序列并行”的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。只看显存节省会忽略通信；只看理论带宽又会忽略启动延迟、拓扑不均匀和流水线气泡。

8 流水线并行

本节路线

本节依次处理：数据张量流水线并行的适用边界、参数梯度优化器状态的显存核算、并行策略由内存通信和利用率共同决定。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$\eta_{\text{pipe}} = \frac{m}{m + p - 1}$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

¹⁹视频画面时间区间：00:55:45–00:56:00。

$$G_7 = \frac{B_7}{C_7 + \lambda R_7 + \epsilon}$$

- B_7 是本节方法带来的质量、吞吐或覆盖收益。
- C_7 是新增计算、显存、通信、数据或标注成本。
- R_7 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_7 \leq C_{\max}, \quad R_7 \leq R_{\max}, \quad Q_7 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

8.1 数据张量流水线并行的适用边界

核心判断。“数据张量流水线并行的适用边界”是“流水线并行”中的一个可检验设计选择，而不是只需记忆的产品名。每种并行都要列出切分维、复制状态和通信原语；端到端收益由通信是否可隐藏、链路拓扑和负载均衡决定。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 20 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“数据张量流水线并行的适用边界”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。


```

409 # B2
    batch_size 83 --- = 128
    num_dim batch_size = data.size(0) = 1024
    local_batch_size data.size(1) = 32
    start_index batch_size = int_divide(batch_size, world_size)
    end_index rank_index = rank * local_batch_size = 32
410 end_index = start_index + local_batch_size
410 data = data[start_index:end_index].to(cuda_if_available(rank))
411
412 # Create MLP parameters params[0], ..., params[num_layers - 1] (each rank has all parameters)
413 params = [get_init_params(num_dim, num_dim, rank) for layer in range(num_layers)]
414 optimizer = torch.optim.Adam(params, lr=1e-3) # Each rank has own optimizer state
415
416 for step in range(num_steps):
417     # Forward pass
418     x = data
419     for param in params:
420         x = x @ param
421         x = F.gelu(x)
422     loss = x.square().mean() # Loss function is average squared magnitude
423
424 # Backward pass

```

图 20: 数据张量流水线并行的适用边界²⁰

8.2 参数梯度优化器状态的显存核算

核心判断。“参数梯度优化器状态的显存核算”是“流水线并行”中的一个可检验设计选择，而不是只需记忆的产品名。把该主题拆成输入、变换、持久状态和输出四部分，再分别记录直接机制指标、资源成本和最终质量；若中间量不按预测变化，应优先检查实现和数据边界。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 21 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“参数梯度优化器状态的显存核算”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

²⁰视频画面时间区间：00:58:30–00:58:45。

```

443 data = generate_sample_data()
444 spawn(tensor_parallelism_main, world_size=4, data=data, num_layers=4)
445
446
447 def tensor_parallelism_main(rank: int, world_size: int, data: tensor, num_layers: int):
448     setup(rank, world_size)
449
450     data = data.to(cuda_if_available(rank)) # All ranks get the data (batch_size x num_dim)
451     batch_size = data.size(0)
452     num_dim = data.size(1)
453     local_num_dim = int_divide(num_dim, world_size) # Shard `num_dim`
454
455     # Create model (each rank gets 1/world_size of the parameters)
456     # | | | |
457     # W0 W1 W2 W3
458     # | | | |
459     params = [get_init_params(num_dim, local_num_dim, rank) for layer in range(num_layers)]
460
461     # Forward pass
462     x = data
463     for layer in range(num_layers):
464         # Compute activations (batch_size x local_num_dim)

```

图 21: 参数梯度优化器状态的显存核算²¹

8.3 并行策略由内存通信和利用率共同决定

核心判断。“并行策略由内存通信和利用率共同决定”是“流水线并行”中的一个可检验设计选择，而不是只需记忆的产品名。把该主题拆成输入、变换、持久状态和输出四部分，再分别记录直接机制指标、资源成本和最终质量；若中间量不按预测变化，应优先检查实现和数据边界。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 22 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

常见失效模式

围绕“并行策略由内存通信和利用率共同决定”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

²¹视频画面时间区间：01:04:00–01:04:15。

```

454
455 # Create model (each rank gets 1/world_size of the parameters)
456 # | | | |
457 # W0 W1 W2 W3
458 # | | | |
459 params = [get_init_params(num_dim, local_num_dim, rank) for layer in range(num_layers)]
460
461 # Forward pass
462 x = data
463 for layer in range(num_layers):
464     # Compute activations (batch_size x local_num_dim)
465     x = x @ params[layer] # Note: this is only on a slice of the parameters
466     x = F.gelu(x)
467
468     # Allocate memory for activations (world_size x batch_size x local_num_dim)
469     activations = [torch.empty(batch_size, local_num_dim, device=cuda_if_available(rank)) for _ in range(w
470
471     # Send activations via all gather
472     dist.all_gather(tensor_list=activations, tensor=x, async_op=False)
473
474     # Concatenate them to get batch_size x num_dim
475     x = torch.cat(activations, dim=1)

```

图 22: 并行策略由内存通信和利用率共同决定²²

8.4 本章小结

“流水线并行”的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。只看显存节省会忽略通信；只看理论带宽又会忽略启动延迟、拓扑不均匀和流水线气泡。

9 组合并行策略

本节路线

本节依次处理：端到端并行训练方案的收束示例、数据并行按 rank 切分 batch 的代码、DDP 反向阶段执行梯度 all-reduce 的代码。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$M_{\text{total}} = M_{\text{param}} + M_{\text{grad}} + M_{\text{opt}} + M_{\text{act}}$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

²²视频画面时间区间：01:06:45–01:07:00。

$$G_8 = \frac{B_8}{C_8 + \lambda R_8 + \epsilon}$$

- B_8 是本节方法带来的质量、吞吐或覆盖收益。
- C_8 是新增计算、显存、通信、数据或标注成本。
- R_8 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_8 \leq C_{\max}, \quad R_8 \leq R_{\max}, \quad Q_8 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

9.1 端到端并行训练方案的收束示例

核心判断。“端到端并行训练方案的收束示例”是“组合并行策略”中的一个可检验设计选择，而不是只需记忆的产品名。把该主题拆成输入、变换、持久状态和输出四部分，再分别记录直接机制指标、资源成本和最终质量；若中间量不按预测变化，应优先检查实现和数据边界。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 23 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“端到端并行训练方案的收束示例”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

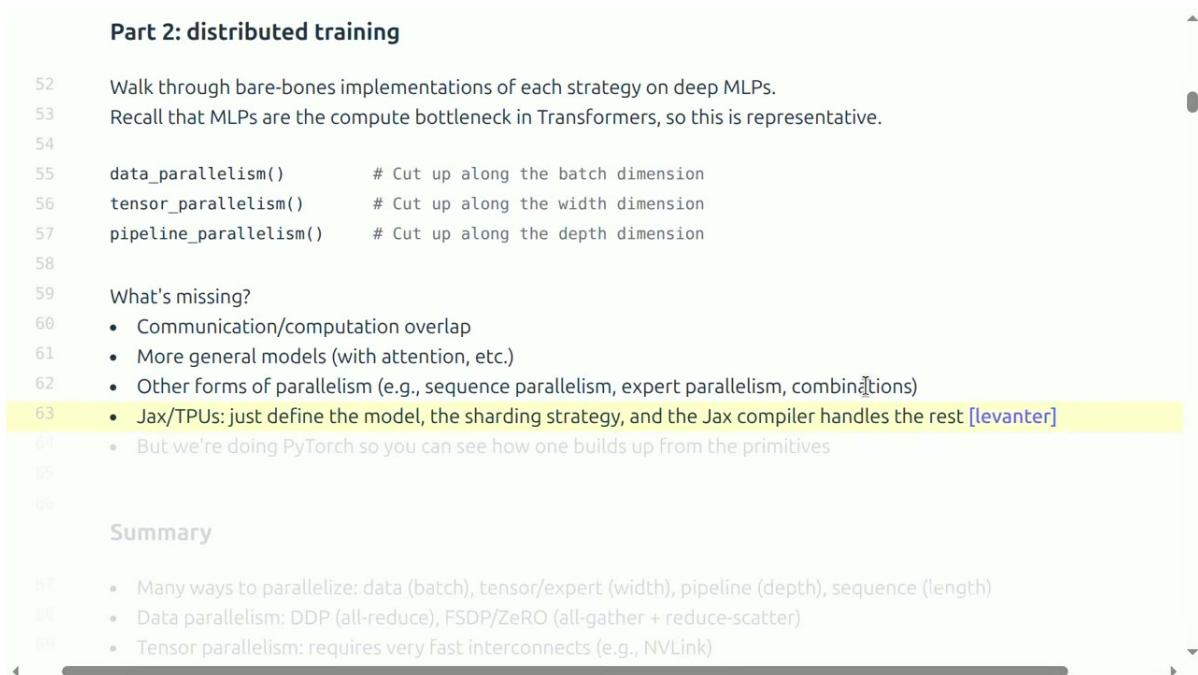


图 23: 端到端并行训练方案的收束示例²³

9.2 数据并行按 rank 切分 batch 的代码

核心判断。“数据并行按 rank 切分 batch 的代码”是“组合并行策略”中的一个可检验设计选择，而不是只需记忆的产品名。每种并行都要列出切分维、复制状态和通信原语；端到端收益由通信是否可隐藏、链路拓扑和负载均衡决定。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 24 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“数据并行按 rank 切分 batch 的代码”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

²³视频画面时间区间：01:18:00–01:18:15。

```

394     return data

    batch_size          = 128
    num_dim              = 1024
    local_batch_size    = int_divide(batch_size, world_size)

def lelism_main(rank: int, world_size: int, data: tensor, num_layers: int, num_steps: int):
    setup(rank, world_size)

399
400     # Get the slice of data for this rank (in practice, each rank should load only its own data)
401     # --- B0 ---
402     # --- B1 ---
403     # --- B2 ---
404     # --- B3 ---
405     batch_size = data.size(0)
406     num_dim = data.size(1)
407     local_batch_size = int_divide(batch_size, world_size)
408     start_index = rank * local_batch_size
409     end_index = start_index + local_batch_size
410     data = data[start_index:end_index].to(cuda_if_available(rank))
411
412     # Create MLP parameters params[0], ..., params[num_layers - 1] (each rank has all parameters)
413     params = [get_init_params(num_dim, num_dim, rank) for layer in range(num_layers)]
414     optimizer = torch.optim.AdamW(params, lr=1e-3) # Each rank has own optimizer state
415

```

图 24: 数据并行按 rank 切分 batch 的代码²⁴

9.3 DDP 反向阶段执行梯度 all-reduce 的代码

核心判断。“DDP 反向阶段执行梯度 all-reduce 的代码”是“组合并行策略”中的一个可检验设计选择，而不是只需记忆的产品名。每种并行都要列出切分维、复制状态和通信原语；端到端收益由通信是否可隐藏、链路拓扑和负载均衡决定。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 25 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

常见失效模式

围绕“DDP 反向阶段执行梯度 all-reduce 的代码”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

²⁴视频画面时间区间：00:57:15–00:57:30。


```

414     optimizer = torch.optim.AdamW(params, lr=1e-3) # Each rank has own optimizer state
415
416     for step in range(num_steps):
417         # Forward pass
418         x = data
419         for param in params:
420             x = x @ param
421             x = F.gelu(x)
422         loss = x.square().mean() # Loss function is average squared magnitude
423
424         # Backward pass
425         loss.backward()
426
427         # Sync gradients across workers (ONLY difference between standard training and DDP)
428         for param in params:
429             dist.all_reduce(tensor=param.grad, op=dist.ReduceOp.AVG, async_op=False)
430
431         # Update parameters
432         optimizer.step()
433
434     print(f"[data_parallelism] Rank {rank}: step = {step}, loss = {loss.item()}, params =
{[summarize tensor(params[laver]) for laver in range(num_lavers)]}". flush=True)

```

图 25: DDP 反向阶段执行梯度 all-reduce 的代码²⁵

9.4 本章小结

“组合并行策略”的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。只看显存节省会忽略通信；只看理论带宽又会忽略启动延迟、拓扑不均匀和流水线气泡。

10 代码化复现：最小机制

下面的片段保留本讲最核心的程序结构，用于把公式和系统过程落到可执行对象。它不是生产实现；实际复现还需补充张量形状断言、数值精度、同步、随机种子、数据版本和端到端测试。

Listing 1: Lecture 7 机制伪代码

```

1 rank = dist.get_rank()
2 world = dist.get_world_size()
3 local_batch = global_batch[rank::world]
4 loss = model(local_batch).loss
5 loss.backward()
6 for p in model.parameters():
7     dist.all_reduce(p.grad)
8     p.grad /= world

```

²⁵视频画面时间区间：01:00:15–01:00:30。

复现检查单

先写正确性基线，再记录资源基线；每次只改变一个机制；同时保存直接中间量和最终指标；对随机算法报告多次运行；对数据与评测保存不可变版本标识。

10.1 本章小结

代码的价值在于暴露状态、顺序和边界。若一个概念无法写成输入、变换、状态、输出和断言，它通常还没有被真正理解。

11 总结与延伸

一页压缩

本讲的主问题是：当单卡显存和计算不再足够时，应怎样切分数据、参数、激活和层，同时控制通信与同步成本？统一答案是：并行策略的本质是决定哪一维被切分、哪一份状态被复制、在哪个边界执行哪一种集合通信。

第一，优先理解机制而不是产品名。模型、硬件、数据和评测不断更新，但张量形状、状态位置、目标函数、通信边界和测量误差仍然是稳定的分析对象。第二，所有优化都要写出代价守恒：减少某一种成本通常会增加另一种成本或风险。第三，把小规模实验推广到大规模之前，必须检查瓶颈是否改变、数据分布是否改变、实现常数是否仍在同一数量级。第四，最可靠的课程笔记应允许读者沿时间轴回到原视频，并沿官方材料找到公式或代码出处。

开放问题

只看显存节省会忽略通信；只看理论带宽又会忽略启动延迟、拓扑不均匀和流水线气泡。后续实验应把这一风险写成可观测量和停止条件，而不是留在讨论部分。

11.1 本章小结

完成本讲后，读者应能用统一语言解释每个主题的输入、状态、变换、输出、资源成本和失败模式，并据此设计一个含基线、消融、误差条和证据记录的最小实验。

12 资料来源与审计边界

- 课程主页: <https://cs336.stanford.edu/>。
- 视频: <https://www.youtube.com/watch?v=Szp0c wdILO Y>。
- Stanford 官方材料: lecture_07.py; 客座讲座若无官方文件则以视频和字幕为主。
- 本地证据: audio.srt、figure_manifest.tsv、figure_verification.txt、teaching_atoms.tsv、numerical_claims.tsv。

12.1 本章小结

本笔记覆盖播放列表中的本讲内容与课程主页当日可取得的官方材料。没有官方讲义的客座讲座不会被伪装成有讲义；讲者口头推测、本笔记分析框架和官方公式在文字中分别标明。